

Commutative Compaction

Chris Douglas
chris_douglas@berkeley.edu
UC Berkeley
Berkeley, California, USA

Joseph M. Hellerstein
hellerstein@berkeley.edu
UC Berkeley
Berkeley, California, USA

Abstract

Table formats optimize data layouts in immutable stores by committing *compactions*. Despite making no logical changes, layout changes conflict with concurrent transactions, forcing costly restarts of both compactions and user transactions. We introduce *compaction maps*, a compact data structure encoding remapping functions to repair layout conflicts and restore commutativity in data lake settings. We implement compaction maps in Apache Iceberg and demonstrate that repairing even large conflicts through large compactions takes less than a second, including I/O costs in three major US clouds.

CCS Concepts

• **Information systems** → **Data management systems**; *Record and block layout*; *Online analytical processing engines*.

Keywords

compaction, table formats, log-structured tables, data lake

ACM Reference Format:

Chris Douglas and Joseph M. Hellerstein. 2026. Commutative Compaction. In *1st International Workshop on Data FORMATS for Modern Architectures and Workloads (FORMATS '26)*, May 31–June 05, 2026, Bengaluru, India. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/3802514.3809174>

1 Introduction

To support efficient reads, open data lake table formats (Apache Iceberg [4], Apache Hudi [3], Delta Lake [6], Lance [1, 18]) rewrite table data in *compactions*. Compactions improve data layout by materializing the result of a full scan, consolidating base data and *delta files* into a new checkpoint. To support immutable stores and concurrent, non-blocking readers, the old layout remains accessible through prior checkpoints and the compacted layout is written as a new set of objects. Compacted layouts can be 2-3x more efficient for readers [9].

Figure 1 illustrates concurrent compaction C and transaction T_2 . These two tasks read the same version of the table, merging $T_0 \dots T_1$ to produce a consistent snapshot. Then they make conflicting writes: T_2 writes new objects with direct references to the files containing rows it read (d_0^+ and d_1^+); C replaces those files with a compacted layout. Only one of T_2 and C can be a successor to T_1 . If C committed after T_2 , it would lose writes. If T_2 committed after C , subsequent readers and writers would follow T_2 's references to the

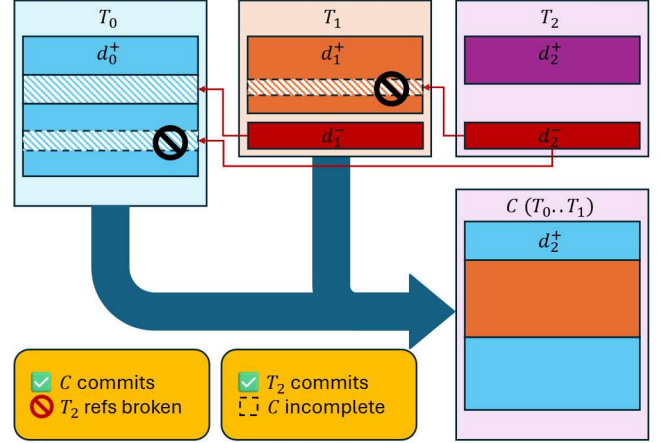


Figure 1: Example compaction. C reorganizes $T_0 \dots T_1$ into a single file; T_2 directly references files in $T_0 \dots T_1$. C and T_2 logically commute, but physically conflict. Compaction maps allow C or T to rewrite position deletes (d_2^-) to reference C , repairing the conflict.

old layout. The conflict requires an expensive reexecution of the losing operation.

The natural solution is to coordinate compaction and transactions, but the *open world* of table formats presents novel coordination challenges: the set of active transactions is unknown to any party, and the only shared medium is a high-latency object store. Traditional techniques—fine-grained latches, page layout indirection, nested top actions [16]—would require coordination through the object store not only for consistency, but also transaction liveness.

Compaction maps let layout-changing compactions commute with concurrent transactional updates, with no coordination beyond the existing snapshot pointer swap. We describe our technique in Section 2 and evaluate its performance in Section 3. Repairing even one million deletes through a large compaction takes less than a second. We briefly discuss related work in Section 4 and conclude in Section 5.

2 Commutative Compaction

Table formats confront a familiar tradeoff between the performance of direct references and the flexibility of an indirection layer that allows reorganization. Direct references can be repaired via pointer swizzling [22]; alternatively, a shared indirection table can gate readers to synchronize changes to referents. Neither of these maps well to table formats, which cannot swizzle direct references in



This work is licensed under a Creative Commons Attribution 4.0 International License. *FORMATS '26, Bengaluru, India*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2635-4/2026/05

<https://doi.org/10.1145/3802514.3809174>

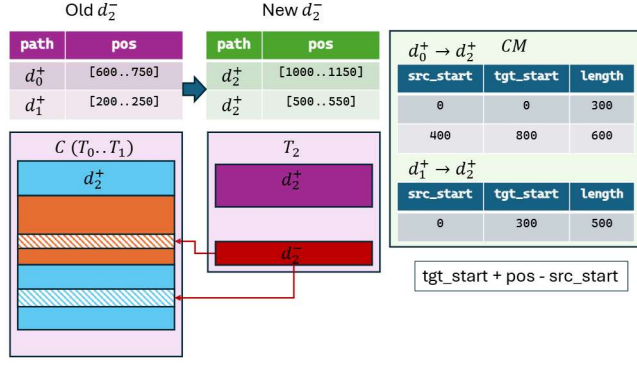


Figure 2: Example rebase. Figure 1 showed a conflicting compaction C and transaction T_2 . Since T_1 deleted rows [300..400] from d_0^+ , the filemap for d_0^+ references two runs in C . To rebase T_2 onto C , only d_2^- is remapped; d_2^+ (from T_2) is unchanged.

immutable objects, nor assume or dictate an intermediate service to intercept reads.

2.1 Compaction Maps

Our solution repairs broken direct references using a compact data structure. We do not track individual references, either via swizzling or indirection. Instead, we record parameters for *remapping functions* that apply to *runs* of rows in the table. During compactions, long runs are transferred from source files to one or more target files. Listing 1 summarizes the schema for *compaction maps*. Since most runs go to the same target file, the `tgt_file` of a run is usually `NULL`; it only overrides the default `def_target` if the compaction splits a file across multiple targets.

```
-- Position ranges within each file mapping
runs(src_file, src_start, src_len, tgt_start, tgt_file)
-- File-level mappings (one row per source file)
file_mappings(src_file, def_target)
-- Positions to remap (from delete file)
positions(src_file, pos)
```

Listing 1: Compaction Map Schema

Writers use compaction maps to repair conflicts before commit; readers ignore them. Assume a transaction T that runs concurrently with a compaction C . If T commits before C , then C can *rebase* T by aligning the referents of T 's position deletes from the old layout to the compacted layout using the remapping functions in the compaction map. Symmetrically, T can perform the same operation to commute with a committed C , if C invalidated the direct references in T to the old layout.

Figure 2 illustrates the repair of the conflict in Figure 1. Our goal is a new state that includes both the change in layout written by C and the updates written by T_2 . T_2 deleted rows from d_0^+ and d_1^+ , which don't exist in the new layout. To remap position $\langle d_0^+, 600 \rangle$, we look up the corresponding range in the file mapping d_0^+ ([400..1000]), recorded in the compaction map as the start offset and length. In the target file written by C , this range starts at position 800, so we align $\langle d_0^+, 600 \rangle$ in the run (i.e., `pos - src_start`, 200) and locate it in the destination (i.e., `tgt_start + 200`, 1000).

The compaction map is *total*, since every position in T_2 's position deletes is in some run, because C copies all live rows and deletes can only target live rows.

Rebasing T on C rewrites only the position deletes d^- to reference C . For every point p in d^- , there is a run r in the compaction map mapping it to a new point in C . The inserts d^+ from T are unmodified. Remapping d^- allows C to commute with a committed T or vice versa.

2.2 Remapping Operation

Listing 2 summarizes pseudocode for the remapping operation. We express this declaratively in SQL, since the remapping implementation selects an algorithm based on the number of regions remapped by the compaction (runs, r) and the number of positions (p) in the position delete file. We detail the policy in Section 3.3.

```
SELECT p.src_file AS src_file,
       p.pos AS src_pos,
       COALESCE(r.tgt_file, fm.def_target) AS dst_file,
       r.tgt_start + (p.pos - r.src_start) AS dst_pos
FROM positions p
JOIN file_mappings fm USING (src_file)
JOIN runs r USING (src_file)
WHERE p.pos >= r.src_start
      AND p.pos < r.src_start + r.src_len
```

Listing 2: Remapping Algorithm (logical)

The remapping operation rewrites the position deletes of the transaction T to reference the compaction C . Whether C or T commits first, the resulting state is the same: T 's deletion vector references C (instead of the logical precursor of C) and T 's inserts are unchanged.

One can rebase more than one transaction before committing C . For example, if C compacts T_i , references among T_j where $j > i$ are still valid; C only invalidates references at or preceding d_i^+ , so C can commit after rebasing all T_j .

3 Evaluation

We evaluate the end-to-end remapping cost in a cloud setting (§3.2) and also the cost of remapping position deletes in microbenchmarks to show the impact of different remapping strategies (§3.3).

3.1 Apache Iceberg

We summarize the relevant layout details in Apache Iceberg, our reference implementation for compaction maps. We modified the compaction operation in Apache Iceberg 1.10.1 implemented in Apache Spark [23] v3.5 and v4.0. Our prototype supports both v2 and v3 tables.

3.1.1 Delete Formats. Deletions in Iceberg are represented either *positionally*—capturing the identity of a deleted row by its immutable location in a file—or by a value-based *equality* predicate, causing subsequent reads to skip every row matching the predicate. Position deletes in Iceberg are represented in two formats: v2 tables use Position Delete Files (PD); v3 tables use Position Delete Vectors (DV). Both encode tombstones at row positions in a file, but PD are separate Parquet files while DV are encoded as roaring bitmaps [10] in a region in Puffin [5] files, an extensible container format that also stores table statistics and sketches. Roaring bitmaps compress

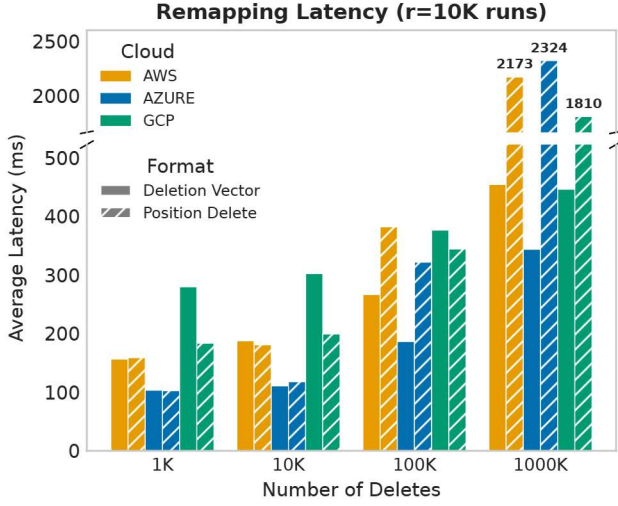


Figure 3: Remapping scalability across clouds and formats.

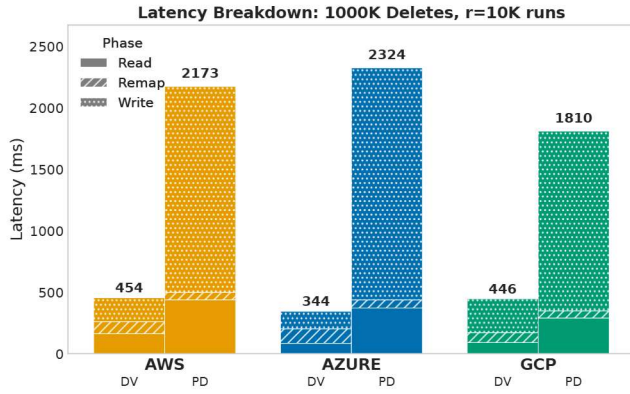


Figure 4: Breakdown of remapping latency of 1000K deletes and 10k runs across clouds and formats.

well with low encode/decode costs, properties that become salient in Section 3.3. We evaluate both representations because v2 tables remain widespread, not because the choice involves a meaningful tradeoff. DV performance dominates PD in every case we evaluated.

3.1.2 Implementation and Compatibility Details. Using Iceberg’s vernacular, our extension supports both *bin-pack* compactions that concatenate unordered updates, and *merge* compactions that interleave updated tuples with the base data. We do not consider compaction maps for rewrites that reorder the data (i.e., SORT and Z-ORDER), since they tend to change the position of most rows, leading to large compaction maps. The limitation is not fundamental: we lack the data needed to tune a policy, so we leave reordering rewrites to future work.

During a compaction, live rows in a source file are remapped to one or more target files, depending on the maximum target file size. Equality predicate deletes are applied during the compaction, omitting all matching runs in the source file. Concurrent equality

deletes trivially commute with compactions, since predicates apply by value not position.

3.1.3 Compaction Map Storage Details. Following conventions in Iceberg, compaction maps are written in Apache Avro [2] format. Each mapping scopes a relocated run to its source file and (default) target file. Each row within a mapping contains a source and target position, length, and possible override of the default target, to handle cases where the compaction splits a source across multiple files. Even without optimization, the format is compact, requiring 2.6KiB for 10 runs and 8.9KiB for 10k runs. Compaction maps are referenced by entries in the *manifest list*¹. Compaction maps could be included in Puffin files, but we store them as separate objects referenced close to the table root. A separate file adds a nominal GC overhead, but including the reference in the manifest list avoids loading larger objects lower in the tree.

Including the reference close to the root avoids another false conflict: the existence of a compaction map allows *SERIALIZABLE* transactions to ignore conflicts if the compaction merely changed the layout of data in the readset. Serializable transactions can be retried without re-validating read conflicts with a new layout.

3.2 Cloud Benchmarks

We evaluated remapping performance in major US clouds². Benchmarks accessed object storage in the same region as the VM to minimize network variability. Figure 3 measures the cost of commuting a transaction with a large compaction (10K runs), including I/O. PD sizes in these experiments range from 1.3 to 2.8MiB, while the roaring bitmap encoding (DV) requires as little as 6KiB for 1000K dense deletes, but 1.2-3.1MiB for sparse patterns. Figure 4 shows that Parquet encoding is responsible for the bulk of the cost in PD files. In v3 Iceberg tables the high encoding costs are not paid; PD files may be read, but new commits write deletion vectors.

To provide a different cross-cut of variables, Figure 5 fixes the cloud, but varies the number of runs in the compaction map, which was fixed in Figure 3. Even with 10K runs in a 1000K file, latency in any cloud never exceeds half a second. Compared to re-running the conflicting compaction or transaction, remapping is always cheaper.

3.3 Remapping Algorithms

Remapping takes as input a set of p positions and r runs and selects a policy based on the shape of these inputs. Unsorted PD files use an *IntervalTree* ($O(p \log r)$) to implement the join when the $O(r)$ build cost is amortized over a large number of positions, but for compactions where $r \gg p$, we sort the positions and fall back to a *MergeJoin*. DV roaring bitmaps are inherently ordered, but a *MergeJoin* ($O(p + r)$) is less efficient for sparse DV (most ranges match nothing) or a small number of ranges (more efficient to match batches of positions per range). For these cases the policy selects an indexed *LoopJoin* strategy ($O(r \log p)$) that takes the start position of each range and uses binary search among sorted positions to

¹In Iceberg, manifest list references manifest files, which reference data files. The manifest list is rewritten for each commit.

²GCP (GCS), AWS (S3), and Azure (ADLSv2) in uswest1, us-west-2, and westus2, respectively. Each cloud was configured with equivalent compute resources: 4 vCPUs and 16 GB RAM, n2-standard-4, m5.xlarge, Standard_D4s_v3. We used OpenJDK 17 on Ubuntu 22.04 LTS across all platforms.

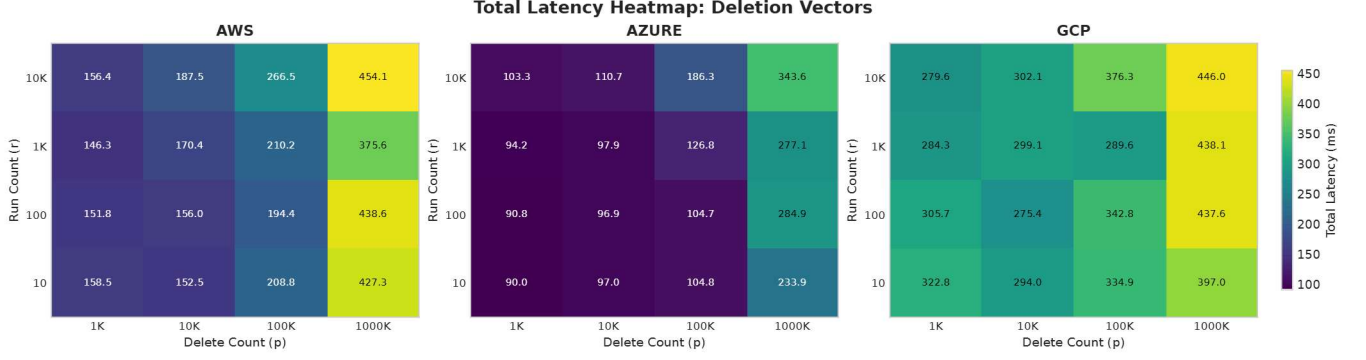


Figure 5: Total latency of compaction including I/O (ms). Varying runs (r) in the compaction map and DV size (p). Figure 3 provides finer detail of top row (10K runs) while 4 decomposes the top-right cell in each cloud.

find all matches to the range. Runs are filtered by min/max position bounds of the delete positions.³

Compactions also coalesce compaction maps when it is sound to do so. If inputs to a compaction include a region with a compaction map, overlapping intervals generate direct references to the new layout; writers do not need to “chain” compaction maps to locate the output region.

The sub-second cost of rebasing a compaction is nominal among Iceberg operations that typically run for minutes or hours. However, comparing remapping latency to the cost of transaction reexecution fails to capture the operational headaches created by compactions. Failing compactions accumulate a backlog under sustained write workloads, motivating smaller, more frequent compactions. But each compaction creates a new commit that conflicts with concurrent transactions, motivating larger compactions, scheduled off-cycle. Compaction maps cut the knot of workarounds and special logic by making compactions commute.

4 Related Work

Compaction maps were inspired by REMIX [24], an LSM index structure that materializes the path of a reader through sorted runs (SSTables). While a REMIX was designed to quickly align a reader and avoid redundant re-execution of an LSM merge, compaction maps operate on the index structure directly. If the table is sorted, then adding “anchor keys” to seek within the compaction map could serve a similar purpose: recording the path that readers take when scanning d^+ through a table state. Compactions do not reclaim storage until the old checkpoint is garbage-collected. Instead of performing the compaction, building a compaction map as an index could improve read performance with much lower storage overhead.

Alternatively, one could embrace indirection in representation and specify deletes by row id to commute without remapping. In columnar formats, including the `row_id` attribute on every query adds a cost, but readers can sometimes avoid it by mapping deletions to intervals and eliding the attribute in rows without matches (§3.3). Iceberg v3 tables include row ids. Where table format data are

materialized on-the-fly [13, 15, 20], it may be less expensive to use row ids than to synthesize direct references.

Data structures that repair and avoid conflicts with committed transactions have many precedents [7, 16, 21]. MV3C [11] partitions and re-executes conflicting regions of a transaction. Morty [8] uses continuations to re-execute subgraphs updated by committed transactions. SIREAD locks in Serializable Snapshot Isolation [12, 19] anticipate and prevent consistency violations in subsequent transactions. Compaction maps anticipate layout conflicts and provide an efficient means of repair.

Compaction maps could also be extended to “invert” delta commits, by rewriting the past commit to reference the *future* compacted state. Inserts can be directly converted to position deletes using the compaction map; the deletes can be used to generate inserts from the old base data. This can reduce the number of compacted states maintained in the table. Common FASTAPPEND transactions only insert new data; converting them to position deletes in the compacted state saves space.

Frameworks that choose when and how to compact table formats solve a related, but orthogonal problem [14]. Compaction maps reduce the cost of interference with concurrent transactions to a negligible overhead on both the compaction and the transactions, eliminating one dimension of this optimization problem.

Delta Lake [6] tombstones are positional, and its compaction commits are already distinguished from data-changing ones, so a compaction-map artifact and a remap step at commit time would suffice. Hudi [3] writes identify records by key rather than position, so there is no positional reference for a compaction map to repair. As with sort and z-order rewrites in Iceberg, these techniques could apply to Hudi’s reclustering operations, though at a coarser granularity than row positions.

5 Conclusion

We introduced and evaluated compaction maps, a compact data structure for commuting compactions in table formats. Repair completes in under a second across three major clouds, even for large conflicts. While we focus on compaction in this short paper, maintenance operations such as deduplication and GDPR (“right to be forgotten”) are closely related and admit similar solutions.

³To tune the join selection policy, we ran JMH [17] benchmarks varying formats, compaction runs, delete counts, clustering sparse/dense, and sortedness (in PD).

References

- [1] 2026. *Lance Table Format - Lance*. <https://lance.org/format/table/>
- [2] Apache Software Foundation. 2026. *Apache Avro*. <https://avro.apache.org/>
- [3] Apache Software Foundation. 2026. *Apache Hudi*. <https://hudi.apache.org/>
- [4] Apache Software Foundation. 2026. *Apache Iceberg*. <https://iceberg.apache.org/>
- [5] Apache Software Foundation. 2026. *Puffin Spec - Apache Iceberg*. <https://iceberg.apache.org/puffin-spec/>
- [6] Michael Armbrust, Tathagata Das, Liwen Sun, Burak Yavuz, Shixiong Zhu, Mukul Murthy, Joseph Torres, Herman Van Hovell, Adrian Ionescu, Alicja Luszczak, Michał Świtakowski, Michał Szafranski, Xiao Li, Takuya Ueshin, Mostafa Mokhtar, Peter Boncz, Ali Ghodsi, Sameer Paranjpye, Pieter Senster, Reynold Xin, and Matei Zaharia. 2020. Delta lake: high-performance ACID table storage over cloud object stores. *Proceedings of the VLDB Endowment* 13, 12 (Aug. 2020), 3411–3424. doi:10.14778/3415478.3415560
- [7] Peter Bailis, Alan Fekete, Michael J Franklin, Ali Ghodsi, Joseph M Hellerstein, and Ion Stoica. 2014. Coordination avoidance in database systems. *Proceedings of the VLDB Endowment* 8, 3 (2014), 185–196.
- [8] Matthew Burke, Florian Suri-Payer, Jeffrey Helt, Lorenzo Alvisi, and Natacha Crooks. 2023. Morty: Scaling Concurrency Control with Re-Execution. In *Proceedings of the Eighteenth European Conference on Computer Systems* (Rome, Italy) (*EuroSys '23*). Association for Computing Machinery, New York, NY, USA, 687–702. doi:10.1145/3552326.3567500
- [9] Jesús Camacho-Rodríguez, Ashvin Agrawal, Anja Gruenheid, Ashit Gosalia, Cristian Petculescu, Josep Aguilar-Saborit, Avriella Floratou, Carlo Curino, and Raghu Ramakrishnan. 2024. LST-Bench: Benchmarking Log-Structured Tables in the Cloud. *Proceedings of the ACM on Management of Data* 2, 1 (2024), 1–26.
- [10] Samy Chambi, Daniel Lemire, Owen Kaser, and Robert Godin. 2016. Better bitmap performance with roaring bitmaps. *Software: practice and experience* 46, 5 (2016), 709–719.
- [11] Mohammad Dashti, Sachin Basil John, Amir Shaikhha, and Christoph Koch. 2017. Transaction Repair for Multi-Version Concurrency Control. In *Proceedings of the 2017 ACM International Conference on Management of Data* (Chicago, Illinois, USA) (*SIGMOD '17*). Association for Computing Machinery, New York, NY, USA, 235–250. doi:10.1145/3035918.3035919
- [12] Alan Fekete, Dimitrios Liarokapis, Elizabeth O'Neil, Patrick O'Neil, and Dennis Shasha. 2005. Making snapshot isolation serializable. *ACM Trans. Database Syst.* 30, 2 (June 2005), 492–528. doi:10.1145/1071610.1071615
- [13] Pascal Ginter and Viktor Leis. 2026. Active Data Lakes: Regaining Physical Data Independence Without Losing Interoperability. (2026).
- [14] Anja Gruenheid, Jesús Camacho-Rodríguez, Carlo Curino, Raghu Ramakrishnan, Stanislav Pak, Sumedh Sakdeo, Lenisha Gandhi, Sandeep K. Singhal, Pooja Nilangekar, and Daniel J. Abadi. 2025. AutoComp: Automated Data Compaction for Log-Structured Tables in Data Lakes. doi:10.1145/3722212.3724430 arXiv:2504.04186 [cs].
- [15] Virajith Jalaparti, Chris Douglas, Mainak Ghosh, Ashvin Agrawal, Avriella Floratou, Srikanth Kandula, Ishai Menache, Joseph Seffi Naor, and Sriram Rao. 2018. Netco: Cache and I/O Management for Analytics over Disaggregated Stores. In *Proceedings of the ACM Symposium on Cloud Computing* (Carlsbad, CA, USA) (*SoCC '18*). Association for Computing Machinery, New York, NY, USA, 186–198. doi:10.1145/3267809.3267827
- [16] Chandrasekaran Mohan, Don Haderle, Bruce Lindsay, Hamid Pirahesh, and Peter Schwarz. 1992. ARIES: A transaction recovery method supporting fine-granularity locking and partial rollbacks using write-ahead logging. *ACM Transactions on Database Systems (TODS)* 17, 1 (1992), 94–162.
- [17] OpenJDK. 2026. *Java Microbenchmark Harness (JMH)*. <https://openjdk.org/projects/code-tools/jmh/>
- [18] Weston Pace, Chang She, Lei Xu, Will Jones, Albert Lockett, Jun Wang, and Raunak Shah. 2025. Lance: Efficient Random Access in Columnar Storage through Adaptive Structural Encodings. *arXiv preprint arXiv:2504.15247* (2025).
- [19] Dan RK Ports and Kevin Grittner. 2012. Serializable Snapshot Isolation in PostgreSQL. *Proceedings of the VLDB Endowment* 5, 12 (2012).
- [20] Kaushik Veeraraghavan, Jason Flinn, Edmund B. Nightingale, and Brian Noble. 2010. quFiles: The right file at the right time. *ACM Trans. Storage* 6, 3, Article 12 (Sept. 2010), 28 pages. doi:10.1145/1837915.1837920
- [21] W.E. Weihl. 1988. Commutativity-based concurrency control for abstract data types. *IEEE Trans. Comput.* 37, 12 (1988), 1488–1505. doi:10.1109/12.9728
- [22] Paul R. Wilson. 1991. Pointer swizzling at page fault time: efficiently supporting huge address spaces on standard hardware. *SIGARCH Comput. Archit. News* 19, 4 (July 1991), 6–13. doi:10.1145/122576.122577
- [23] Matei Zaharia, Mosharaf Chowdhury, Michael J Franklin, Scott Shenker, and Ion Stoica. 2010. Spark: Cluster computing with working sets. In *2nd USENIX workshop on hot topics in cloud computing (HotCloud 10)*.
- [24] Wenshao Zhong, Chen Chen, Xingbo Wu, and Song Jiang. 2021. REMIX: Efficient range query for LSM-trees. In *19th USENIX Conference on File and Storage Technologies (FAST 21)*. 51–64.